
Đại số Bool (Boolean Algebra)

Đại số Bool (Boolean Algebra)

- ❑ Được George Boole phát triển năm 1854
- ❑ Claude Shannon áp dụng vào thiết kế mạch điện năm 1938
- ❑ Cơ sở cho thiết kế mạch số trong máy tính
- ❑ Làm việc với tập $\{0, 1\}$

Đại số Bool: Các phép toán cơ bản

□ Phủ định (Complement): $\bar{x}$

□ $\bar{0} = 1, \bar{1} = 0$

□ Tổng Bool (Boolean Sum): $x + y$

□ $1+1=1, 1+0=1, 0+1=1, 0+0=0$

□ Tích Bool (Boolean Product): $x \cdot y$

□ $1 \cdot 1=1, 1 \cdot 0=0, 0 \cdot 1=0, 0 \cdot 0=0$

□ Thứ tự ưu tiên

□ Phủ định $\rightarrow$ Tích $\rightarrow$ Tổng

Đại số Bool: Các phép toán cơ bản

□ Ví dụ 1: Tính $1 \cdot \bar{0} + \overline{(0 + 1)}$

□ Ví dụ 2: Tính $\bar{x}\bar{y} + x\bar{y}$ với $x=1, y=0$

Hàm Bool

□ Biến Bool: chỉ nhận giá trị 0 hoặc 1

□ Hàm Bool bậc n trên tập $B = \{0, 1\}$

□ Hàm từ $B^n \rightarrow B$

□ $B^n = \{(x_1, x_2, \dots, x_n) \mid x_i \in B\}$

x	y	$F(x, y)$
1	1	0
1	0	1
0	1	0
0	0	0

□ Số hàm Bool bậc n

□ Bậc 1: 4 hàm

□ Bậc 2: 16 hàm

□ Bậc 3: 256 hàm

Bậc	Số hàm
1	4
2	16
3	256
4	65,536
5	4,294,967,296
6	18,446,744,073,709,551,616

Hàm Bool

□ Ví dụ 16 Hàm Bool bậc 2

x	y	F_1	F_2	F_3	F_4	F_5	F_6	F_7	F_8	F_9	F_{10}	F_{11}	F_{12}	F_{13}	F_{14}	F_{15}	F_{16}
1	1	1	1	1	1	1	1	1	1	0	0	0	0	0	0	0	0
1	0	1	1	1	1	0	0	0	0	1	1	1	1	0	0	0	0
0	1	1	1	0	0	1	1	0	0	1	1	0	0	1	1	0	0
0	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0

□ Biểu thức Bool

- $0, 1, x_1, x_2, \dots, x_n$ là các biểu thức Bool;
- Nếu E_1 và E_2 là các biểu thức Bool, thì E_1 , $(E_1 E_2)$, và $(E_1 + E_2)$ là các biểu thức Bool.

Hàm Bool

□ Biểu thức Bool

x	y	z	xy	$\bar{z}$	$F(x, y, z) = xy + \bar{z}$
1	1	1	1	0	1
1	1	0	1	1	1
1	0	1	0	0	0
1	0	0	0	1	1
0	1	1	0	0	0
0	1	0	0	1	1
0	0	1	0	0	0
0	0	0	0	1	1

Hàm Bool

- Hàm F và G trên n biến là bằng nhau nếu và chỉ nếu $F(b_1, b_2, \dots, b_n) = G(b_1, b_2, \dots, b_n)$ với mọi $b_1, b_2, \dots, b_n \in B$.
- Hai biểu thức Bool của cùng một hàm được gọi là Tương đương nhau.
- $\bar{F}$: Hàm bù của hàm F
 - $\bar{F}(x_1, \dots, x_n) = \overline{F(x_1, \dots, x_n)}$.
- Cho F và G là các hàm Bool bậc n
 - Hàm $F + G$ và FG được định nghĩa như sau:
 - $(F + G)(x_1, \dots, x_n) = F(x_1, \dots, x_n) + G(x_1, \dots, x_n)$,
 - $(FG)(x_1, \dots, x_n) = F(x_1, \dots, x_n)G(x_1, \dots, x_n)$.

Các Đẳng thức Bool

□ Chứng minh luật phân phối

$$\square x(y + z) = xy + xz$$

$$\square x + yz = (x + y)(x + z)$$

□ Chứng minh luật nuốt

$$\square x(x + y) = x$$

Đẳng thức	Tên gọi
$\overline{\overline{x}} = x$	Luật phủ định kép
$x + x = x$ $x \cdot x = x$	Luật lũy đẳng
$x + 0 = x$ $x \cdot 1 = x$	Luật đồng thức
$x + 1 = 1$ $x \cdot 0 = 0$	Luật trội
$x + y = y + x$ $xy = yx$	Luật giao hoán
$x + (y + z) = (x + y) + z$ $x(yz) = (xy)z$	Luật kết hợp
$x + yz = (x + y)(x + z)$ $x(y + z) = xy + xz$	Luật phân phối
$\overline{(xy)} = \overline{x} + \overline{y}$ $\overline{(x + y)} = \overline{x} \overline{y}$	Luật De Morgan
$x + xy = x$ $x(x + y) = x$	Luật nuốt
$x + \overline{x} = 1$	Luật đơn vị
$x\overline{x} = 0$	Luật Không

Biểu diễn hàm Bool

□ Biến tử (Literal)

- Biến Bool hoặc phủ định của nó

- Ví dụ: $x, \bar{x}, y, \bar{y}, z, \bar{z}$

□ Tiểu hạng (Minterm) của n biến:

- Tích của n biến tử, mỗi biến xuất hiện đúng 1 lần

- Ví dụ với 3 biến: $xyz, \bar{x}yz, x\bar{y}\bar{z}, \dots$

□ Tiểu hạng có giá trị 1 tại đúng 1 bộ giá trị

- $xyz = 1$ khi $x=1, y=1, z=1$

- $\bar{x}_1x_2\bar{x}_3x_4x_5$ khi?

Biểu diễn hàm Bool

□ Tổng-các-Tích (TCT)

- Biểu diễn hàm Bool = Tổng các tiểu hạng
- Mọi hàm Bool đều có thể biểu diễn thành TCT

□ Phương pháp

- Lập bảng chân trị
- Xác định các hàng có giá trị 1
- Viết các tiểu hạng tương ứng
- Lấy tổng các tiểu hạng

Biểu diễn TCT

□ Ví dụ: $F(x,y,z) = (x+y)\bar{z}$

□ Cách 1: Sử dụng các luật đẳng thức

□ Cách 2: Lập bảng chân trị

Tính đầy đủ hàm

- Tập toán tử đầy đủ hàm:
 - Có thể dùng để biểu diễn mọi hàm Bool

- Các tập đầy đủ hàm:
 - $\{\cdot, +, \bar{}\}$ - tập cơ bản
 - $\{\cdot, \bar{}\}$ vì $x+y = \bar{x} \cdot \bar{y}$
 - $\{+, \bar{}\}$ vì $xy = \bar{x} + \bar{y}$

- Lưu ý: $\{+, \cdot\}$ KHÔNG đầy đủ
 - Không thể biểu diễn $\bar{x}$

Toán tử NAND và NOR

□ Toán tử NAND: $x \mid y$

□ $1 \mid 1 = 0, 1 \mid 0 = 1, 0 \mid 1 = 1, 0 \mid 0 = 1$

□ $\bar{x} = x \mid x$

□ $xy = (x \mid y) \mid (x \mid y)$

□ Toán tử NOR: $x \downarrow y$

□ $1 \downarrow 1 = 0, 1 \downarrow 0 = 0, 0 \downarrow 1 = 0, 0 \downarrow 0 = 1$

□ $\bar{x} = x \downarrow x$

□ $x + y = (x \downarrow y) \downarrow (x \downarrow y)$

□ $\{\mid\}$ và $\{\downarrow\}$ đều là tập đầy đủ hàm

Toán tử NAND và NOR

□ Ví dụ 1: Biểu diễn $\bar{x}$ bằng NAND

□ Ví dụ 2: Biểu diễn $x+y$ bằng NAND

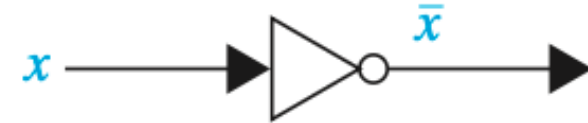
□ Ví dụ 3: $F(x,y) = xy + \bar{x}\bar{y}$

□ Biểu diễn từng thành phần bằng NAND

Cổng Lô gích

□ Bộ đảo Inverter (Cổng NOT)

□ Đầu vào: $x \rightarrow$ Đầu ra: $\bar{x}$



□ Cổng Hoặc (OR)

□ Đầu vào: $x, y \rightarrow$ Đầu ra: $x+y$

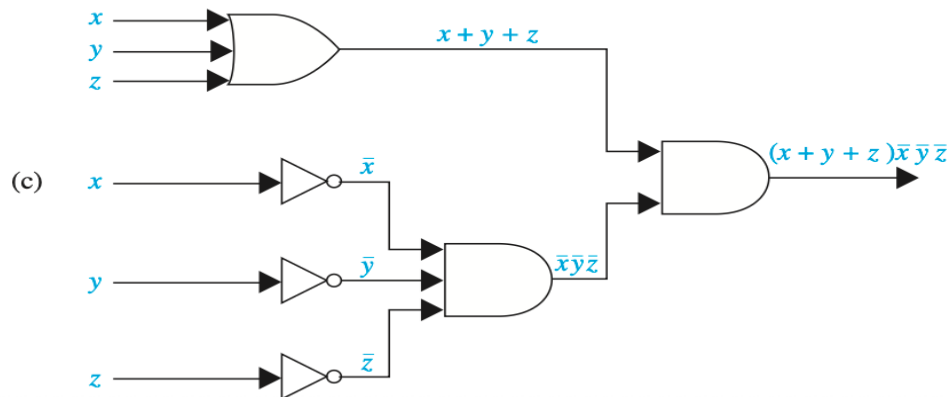
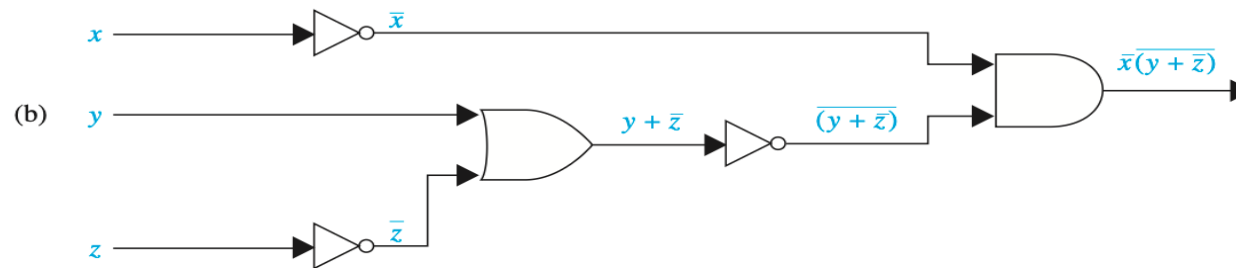
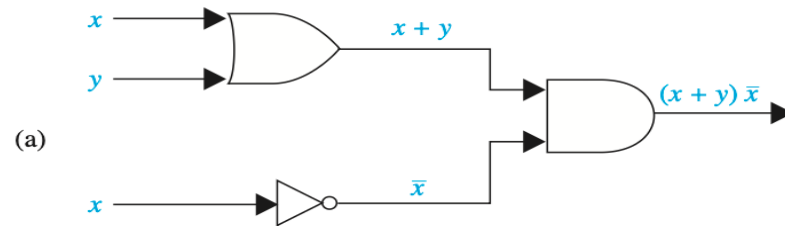


□ Cổng Và (AND):

□ Đầu vào: $x, y \rightarrow$ Đầu ra: xy

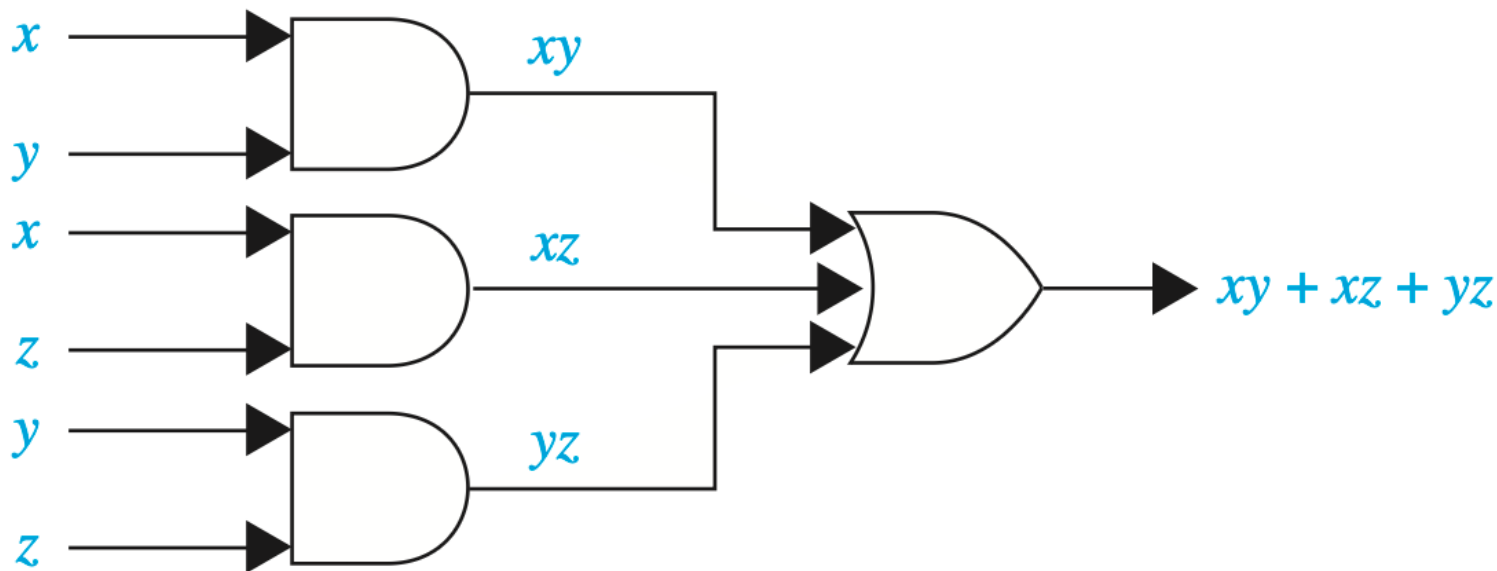


Thiết kế mạch



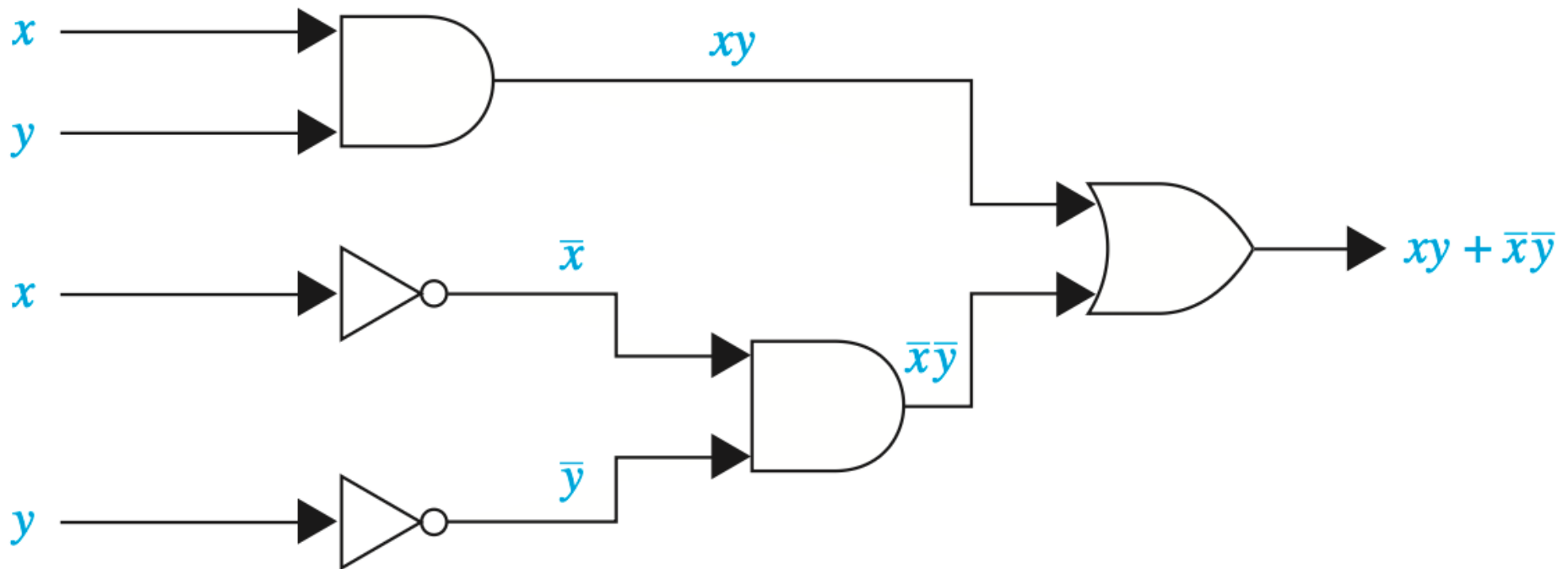
Mạch biểu quyết

- Bài toán: 3 người bỏ phiếu (x, y, z)
 - Đề xuất thông qua nếu ≥ 2 phiếu đồng ý
- Hàm Bool:
 - $F(x,y,z) = 1$ nếu ≥ 2 người đồng ý



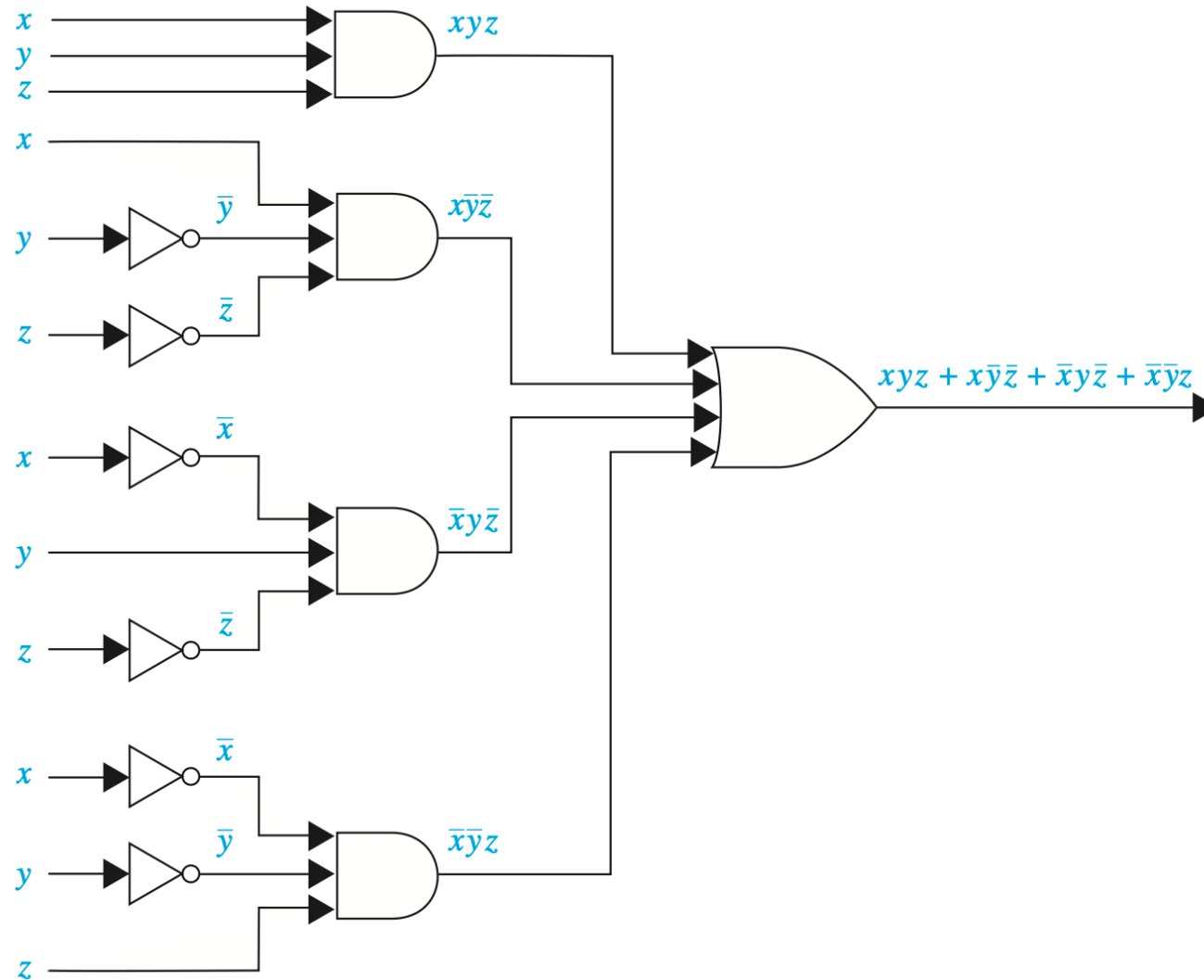
Mạch điều khiển

□ Điều khiển 1 bóng đèn dùng 2 công tắc



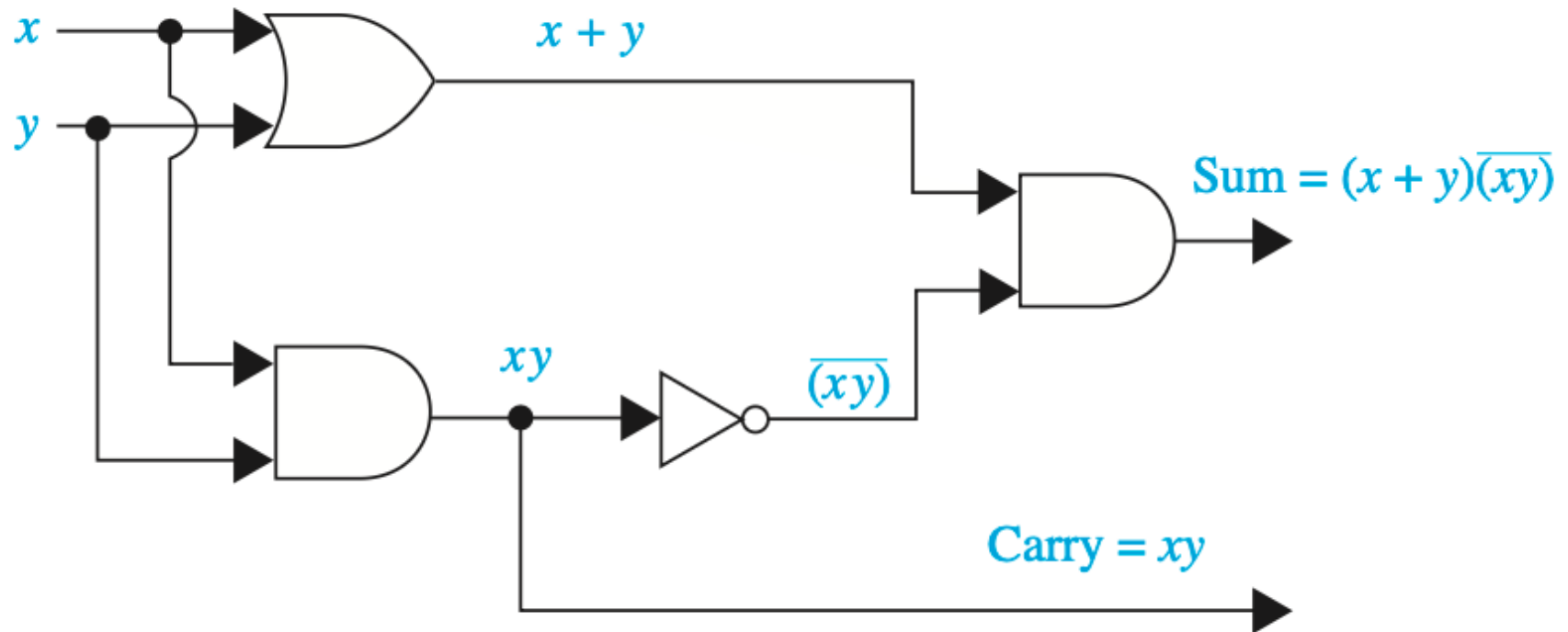
Mạch điều khiển

□ Điều khiển 1 bóng đèn dùng 3 công tắc



Bộ bán cộng (Half Adder)

- Cộng 2 bit x và y :
 - Sum bit (s): $x \oplus y$
 - Carry bit (c): xy

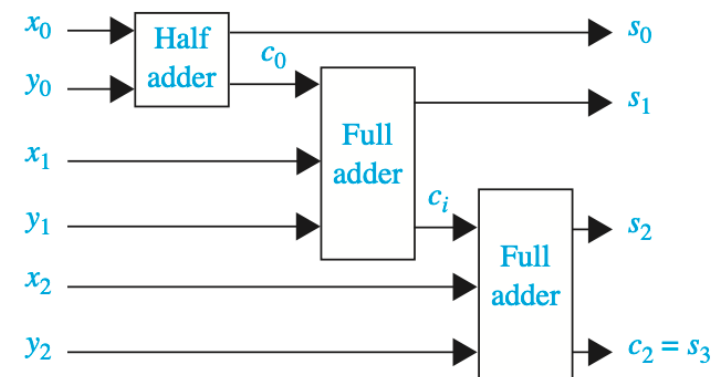
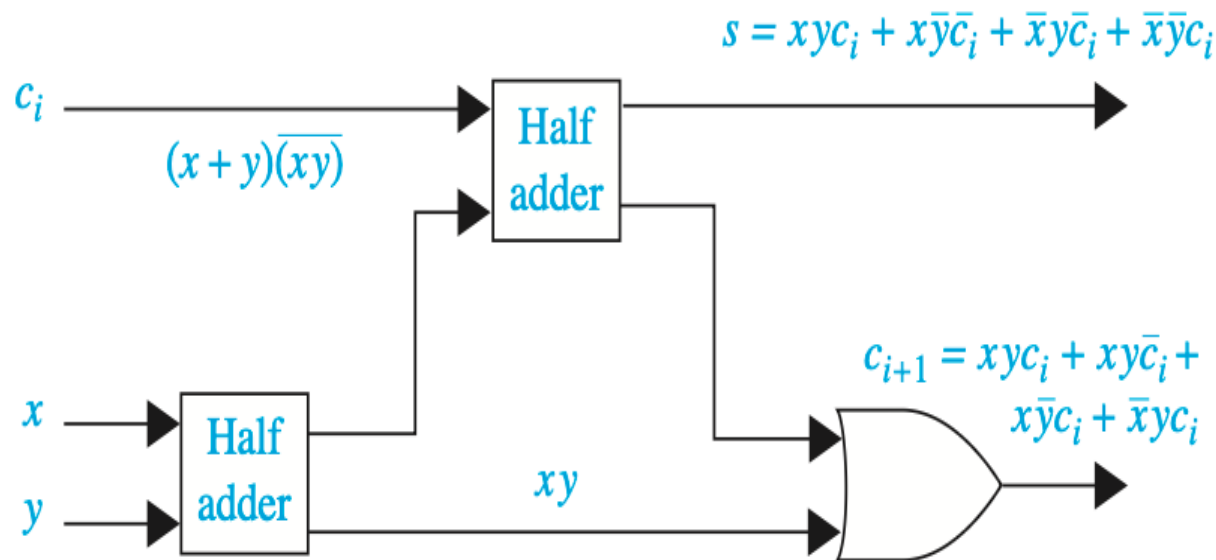


Bộ cộng hoàn chỉnh (Full Adder)

□ Cộng 2 bit x , y và carry c_i :

□ Sum: $s = x \oplus y \oplus c_i$

□ Carry out: c_{i+1}



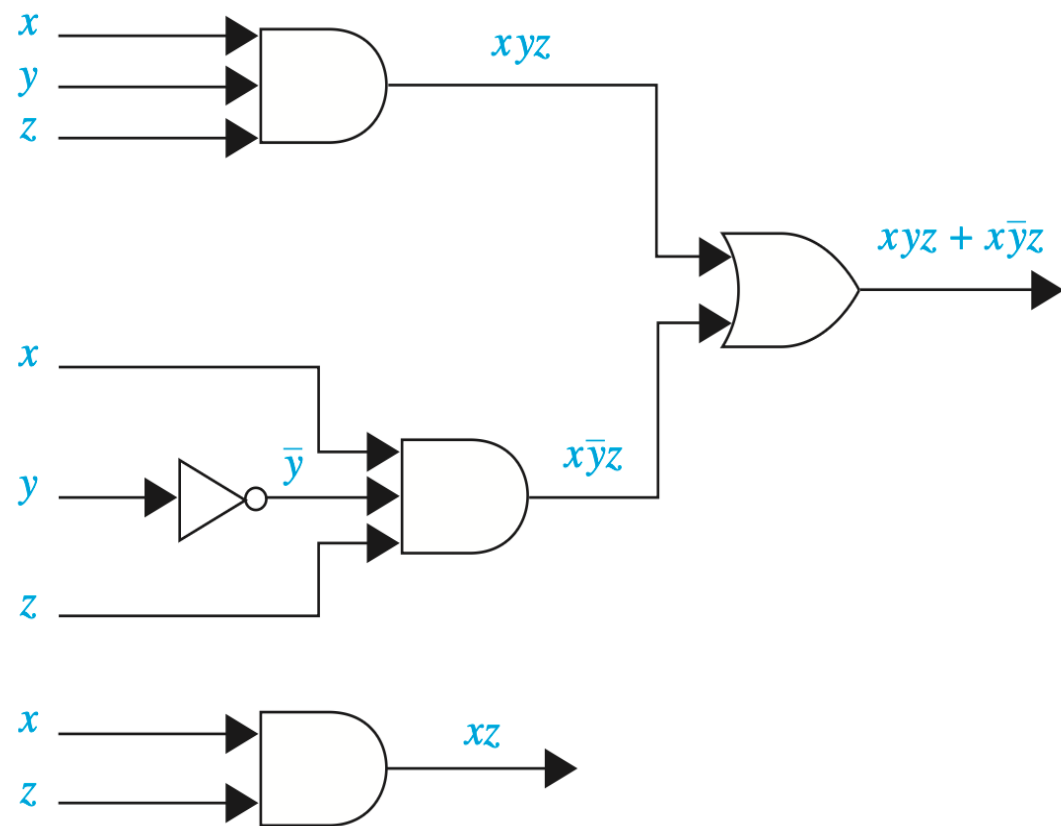
Tối ưu mạch

❑ Hai mạch cho cùng 1 kết quả đầu ra

❑ Các thuật toán

❑ Bản đồ Karnaugh

❑ Quine–McCluskey



Thuật toán (Algorithm)

Thuật toán: *Định nghĩa*

*“Một **thuật toán** là một chuỗi các bước (chỉ dẫn) chính xác để thực hiện một phép tính toán hoặc giải quyết một bài toán.”*

□ *Ví dụ: Thuật toán tìm phân tử lớn nhất trong một chuỗi hữu hạn*

procedure max($a_1, a_2, \dots, a_n$: các số nguyên)

$max := a_1$

for $i := 2$ **to** n

if $max < a_i$ **then** $max := a_i$

return max { max là phân tử lớn nhất}

Thuật toán: *Các đặc trưng*

- ❑ **Đầu vào (Input):** Thuật toán có các giá trị đầu vào thuộc một tập xác định.
- ❑ **Đầu ra (Output):** từ mỗi tập giá trị đầu vào, thuật toán tạo ra các giá trị đầu ra thuộc một tập xác định. Các giá trị này có thể là lời giải cho bài toán mà thuật toán đang giải quyết.
- ❑ **Tính xác định (Definiteness):** Các bước của thuật toán được xác định một cách chính xác.
- ❑ **Tính chính xác (Correctness):** Thuật toán cần tạo ra các kết quả chính xác với mỗi tập đầu vào.
- ❑ **Tính hữu hạn (Finiteness):** Thuật toán cần tạo ra các giá trị đầu ra sau một số hữu hạn (có thể lớn) các bước đối với mọi tập đầu vào.
- ❑ **Tính hiệu quả (Effectiveness):** Các bước của thuật toán cần được thực hiện một cách chính xác trong một khoảng thời gian hữu hạn.
- ❑ **Tính tổng quát (Generality):** Thuật toán cần khả dụng cho mọi bài toán thuộc cùng một dạng mong muốn, chứ không chỉ áp dụng cho một tập đặc biệt các giá trị đầu vào.

Thuật toán: Ví dụ

□ Thuật toán tìm kiếm tuyến tính

procedure linearSearch($x, a_1, a_2, \dots, a_n$)

// x : số nguyên, $a_1, a_2, \dots, a_n$: các số nguyên phân biệt)

$i := 1$

while ($i \leq n$ **and** $x \neq a_i$)

$i := i + 1$

if $i \leq n$ **then** $location := i$

else $location := 0$

return $location$ { $location$ là chỉ số của số hạng bằng x ,
hoặc là 0 nếu không tìm thấy x }

Thuật toán: Ví dụ

□ *Thuật toán tìm kiếm nhị phân*

procedure `binarySearch`($x, a_1, a_2, \dots, a_n$)

// x : số nguyên, $a_1, a_2, \dots, a_n$: các số nguyên tăng dần

$i := 1$ *{ i là điểm nút trái của khoảng tìm kiếm}*

$j := n$ *{ j là điểm nút phải của khoảng tìm kiếm}*

while $i < j$

$m := \lfloor (i + j) / 2 \rfloor$

if $x > a_m$ **then** $i := m + 1$

else $j := m$

if $x = a_i$ **then** $location := i$

else $location := 0$

return $location$ *{ $location$ là chỉ số dưới của số hạng bằng x , hoặc là 0 nếu không tìm thấy x }*

Thuật toán: Ví dụ

□ *Thuật toán sắp xếp nổi bọt*

procedure bubbleSort($a_1, \dots, a_n$: các số thực với $n \geq 2$)

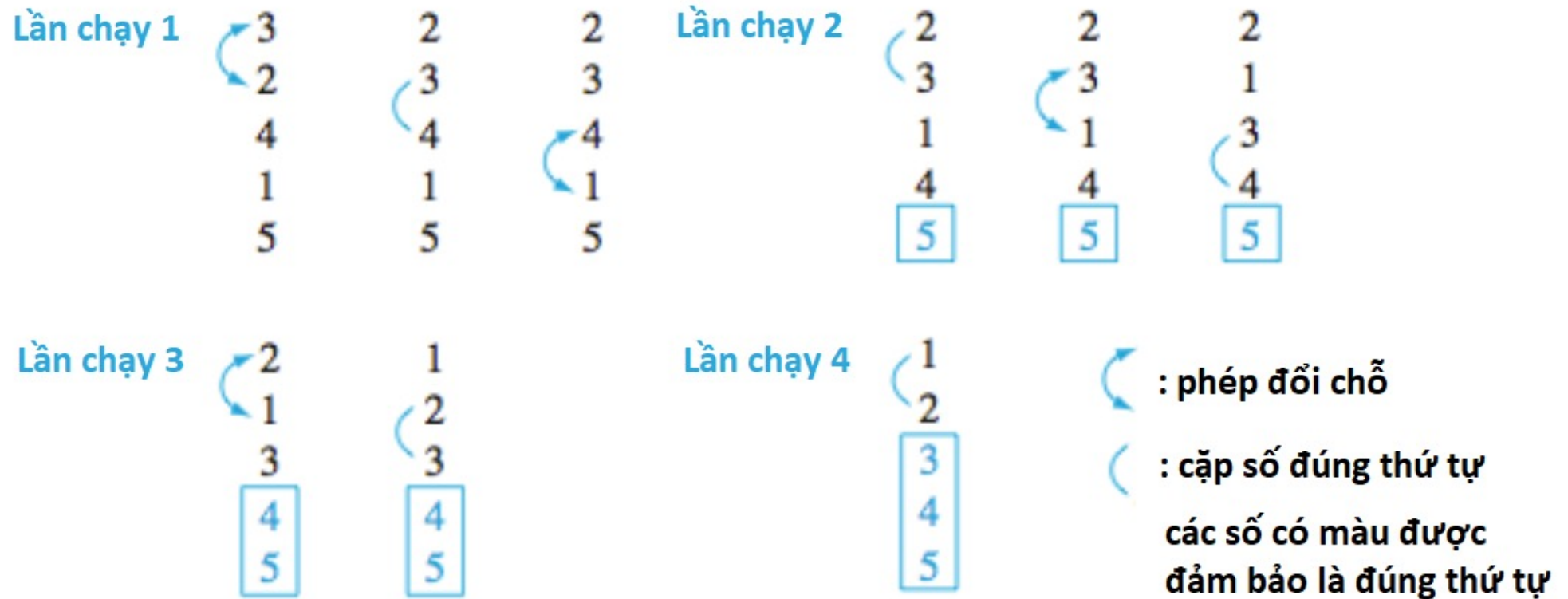
for $i := 1$ **to** $n - 1$

for $j := 1$ **to** $n - 1$

if $a_j > a_{j+1}$ **then** đổi chỗ a_j và a_{j+1}

$\{a_1, \dots, a_n$ được sắp xếp giá trị tăng dần $\}$

Thuật toán: Ví dụ



Hình 1: Các bước của thuật toán sắp xếp nổi bọt

Thuật toán: Ví dụ

□ *Thuật toán sắp xếp chèn*

procedure insertionSort($a_1, a_2 \dots, a_n$: các số thực với $n \geq 2$)

for $j := 2$ **to** n

$i := 1$

while $a_j > a_i$

$i := i + 1$

$m := a_j$

for $k := 0$ **to** $j - i - 1$

$a_{j-k} := a_{j-k-1}$

$a_i := m$

$\{a_1, \dots, a_n$ được sắp xếp giá trị tăng dần}

Thuật toán tham lam (Greedy algorithm)

- ❑ Giải quyết các vấn đề tối ưu hóa (Optimization problems)
 - ❑ Bài toán đổi tiền n xu
 - ❑ Dùng đồng 25 xu, 10 xu, 5 xu, 1 xu
 - ❑ Dùng ít đồng tiền nhất có thể
- ❑ Chọn giải pháp tốt nhất tại mỗi bước thay vì xem xét toàn bộ chuỗi các bước có thể cho lời giải tối ưu nhất

Thuật toán tham lam: Ví dụ

□ *Thuật toán đổi tiền tham lam*

procedure change ($c_1, c_2 \dots, c_r, n$)

// $c_1, c_2 \dots, c_r$: các mệnh giá của xu, với $c_1 > c_2 > \dots > c_r$; n : số nguyên dương

for $i := 1$ **to** r

$d_i := 0$ { d_i đếm số lượng xu mệnh giá c_i được sử dụng}

while $n \geq c_i$

$d_i := d_i + 1$ {thêm một xu mệnh giá c_i }

$n := n - c_i$

{ d_i là số lượng xu thuộc mệnh giá c_i trong tiền đổi với $i = 1, 2, \dots, r$ }

Thuật toán tham lam: Ví dụ

- ❑ Thảo luận trên lớp
 - ❑ Bổ đề 1 – Trang 199

Độ tăng của hàm

- **Khái niệm O-lớn (Big-O):** Cho f và g là các hàm từ tập số nguyên hoặc tập số thực đến tập số thực. Ta nói $f(x)$ là $O(g(x))$ nếu tồn tại hằng số C và k sao cho:

$$|f(x)| \leq C|g(x)|, \forall x > k$$

[Đọc là “ $f(x)$ là O-lớn (big-O) theo $g(x)$.”]

- **Chú ý:** Dễ thấy, $f(x)$ tăng chậm hơn một bội số cố định của $g(x)$ khi x tăng tới vô hạn.
 - C và k được gọi là “chứng cứ (witnesses)”

Độ tăng của hàm

- ❑ **Chú ý:** $f(x)$ là $O(g(x))$ có thể được viết là $f(x) = O(g(x))$.
 - ❑ Dấu bằng ở đây không thể hiện phép bằng thực sự.
 - ❑ Có thể viết là $f(x) \in O(g(x))$ bởi vì $O(g(x))$ biểu diễn một tập các hàm là $O(g(x))$.

- ❑ Khi $f(x)$ là $O(g(x))$, và $h(x)$ là một hàm có giá trị tuyệt đối lớn hơn $g(x)$ với giá trị x đủ lớn,
 - ❑ Suy ra $f(x)$ là $O(h(x))$.
 - ❑ Hàm $g(x)$ trong mỗi quan hệ $f(x)$ là $O(g(x))$ có thể được thay thế bằng một hàm với giá trị tuyệt đối lớn hơn.

Độ tăng của hàm: Ví dụ

□ Chứng minh:

$$f(x) = x^2 + 2x + 1 \text{ là } O(x^2)$$

□ Lời giải:

- Ta thấy rằng khi $x > 1$ ta có:

$$0 \leq x^2 + 2x + 1 \leq x^2 + 2x^2 + x^2 = 4x^2$$

Do đó, chúng ta có thể chọn $C = 4$ và $k = 1$ để thấy rằng $f(x)$ là $O(x^2)$.
Nghĩa là, $f(x) = x^2 + 2x + 1 < 4x^2$ với mọi $x > 1$.

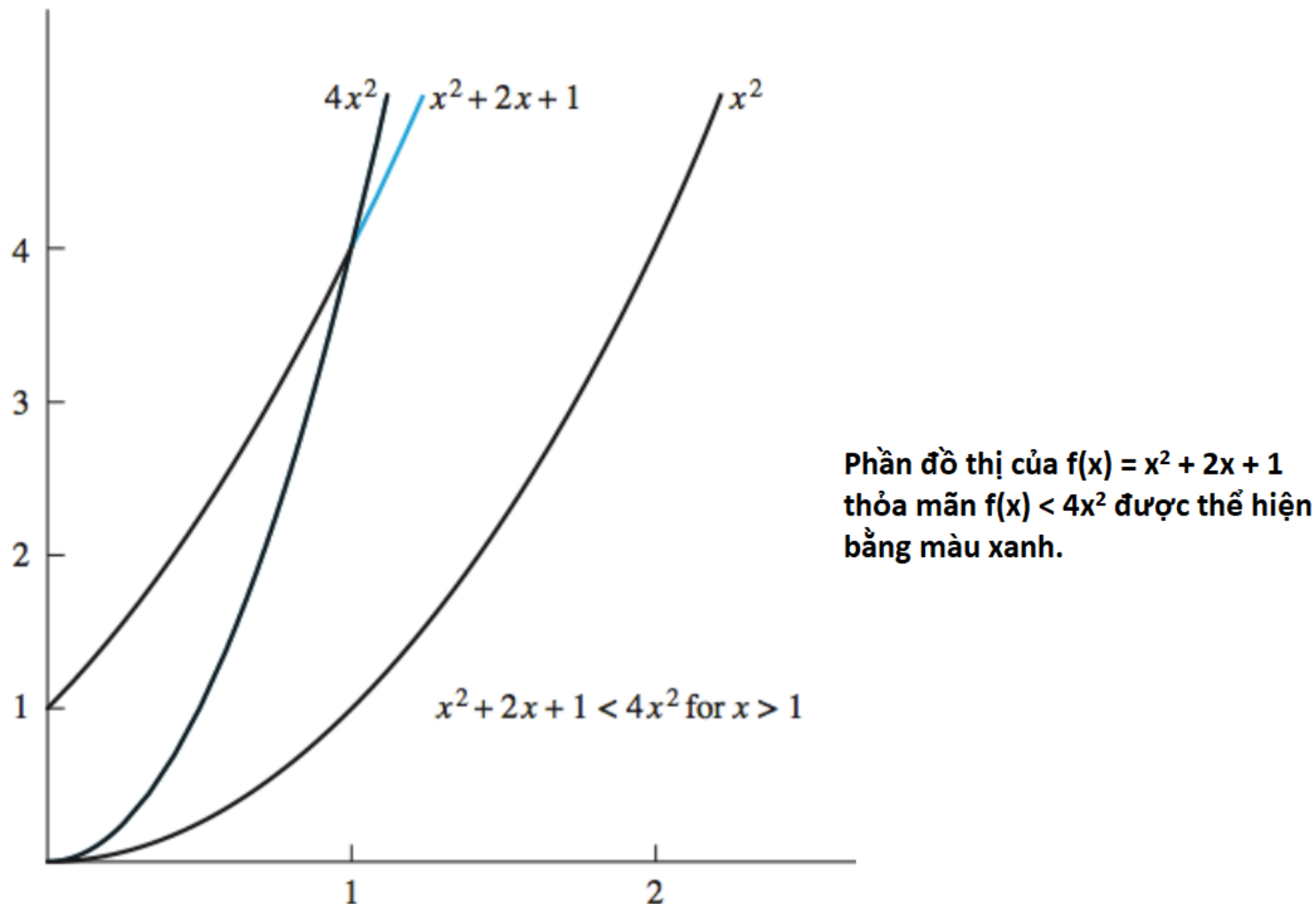
(Lưu ý rằng không cần sử dụng giá trị tuyệt đối ở đây vì tất cả các hàm trong đẳng thức đều dương khi x dương.)

- Ngoài ra, với $x > 2$, chúng ta có $2x \leq x^2$ và $1 \leq x^2$, do đó, với $x > 2$ ta có:

$$0 \leq x^2 + 2x + 1 \leq x^2 + x^2 + x^2 = 3x^2$$

Do đó, $C = 3$ và $k = 2$ cũng là “chứng cứ/witnesses” cho mối quan hệ $f(x)$ là $O(x^2)$.

Độ tăng của hàm: Ví dụ



Hình 1: Hàm $x^2 + 2x + 1$ là $O(x^2)$.

Độ tăng của hàm: Ví dụ

□ Chứng minh:

- Đặt $f(x) = a_n x^n + a_{n-1} x^{n-1} + \dots + a_1 x + a_0$, với $a_0, a_1, \dots, a_{n-1}, a_n$ là các số thực. Khi đó, $f(x)$ là $O(x^n)$.

□ Lời giải:

- Sử dụng bất đẳng thức tam giác (xem Bài tập 7, mục 1.8) với $x > 1$, ta có:

$$\begin{aligned} |f(x)| &= |a_n x^n + a_{n-1} x^{n-1} + \dots + a_1 x + a_0| \\ &\leq |a_n| x^n + |a_{n-1}| x^{n-1} + \dots + |a_1| x + |a_0| \\ &= x^n \left(|a_n| + \frac{|a_{n-1}|}{x} + \dots + \frac{|a_1|}{x^{n-1}} + \frac{|a_0|}{x^n} \right) \\ &\leq x^n (|a_n| + |a_{n-1}| + \dots + |a_1| + |a_0|) \end{aligned}$$

- Qua đó,

$$|f(x)| \leq C x^n,$$

với $C = |a_n| + |a_{n-1}| + \dots + |a_0|$ với mọi $x > 1$. Do đó, $C = |a_n| + |a_{n-1}| + \dots + |a_0|$ và $k = 1$ cho thấy $f(x)$ là $O(x^n)$.

Độ tăng của hàm: Ví dụ

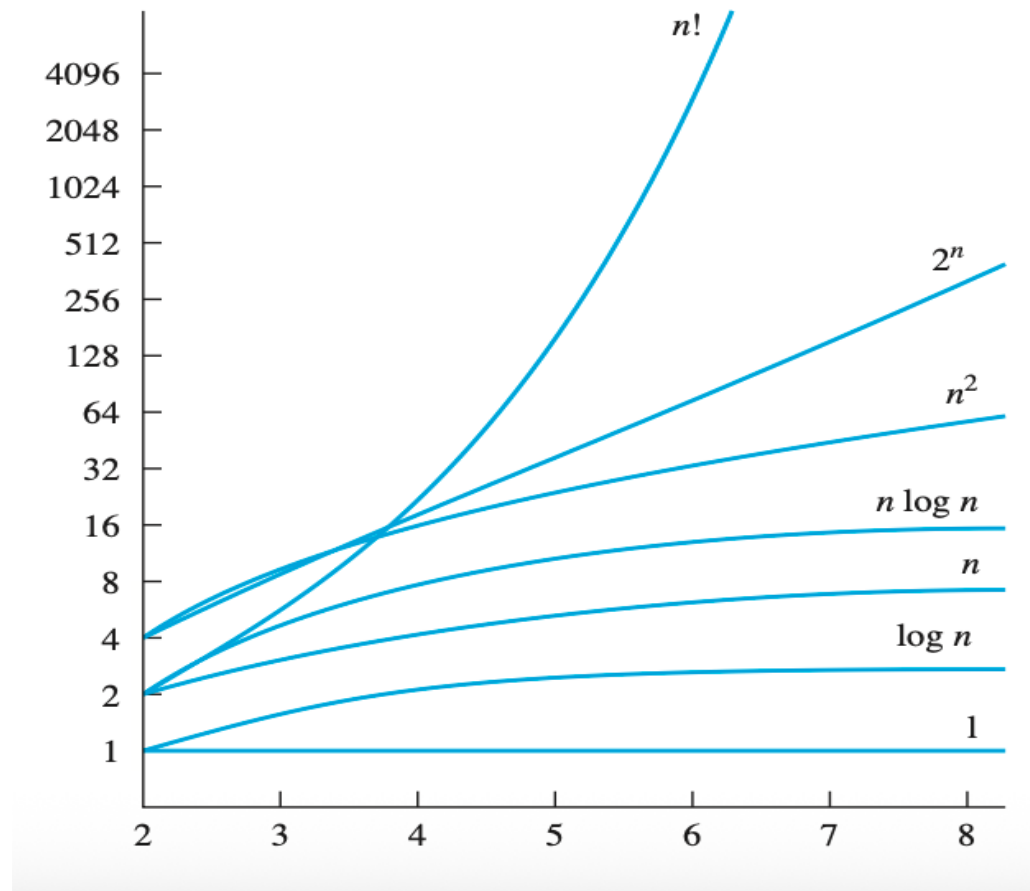
□ CMR

□ $f(n) = 1 + 2 + \dots + n$ là $O(n^2)$

□ $f(n) = n!$ là $O(n^n)$

□ $f(n) = n$ là $O(2^n)$

○ $f(n) = \log(n)$ là $O(n)$



Độ tăng của hàm

□ Định lý:

- Giả sử $f_1(x)$ là $O(g_1(x))$ và $f_2(x)$ là $O(g_2(x))$
 - Thì $(f_1 + f_2)(x)$ là $O(\max(|g_1(x)|, |g_2(x)|))$.

- Giả sử $f_1(x)$ và $f_2(x)$ đều là $O(g(x))$
 - Thì $(f_1 + f_2)(x)$ là $O(g(x))$.

- Giả sử $f_1(x)$ là $O(g_1(x))$ và $f_2(x)$ là $O(g_2(x))$
 - Thì $(f_1 f_2)(x)$ là $O(g_1(x)g_2(x))$.

Độ tăng của hàm: Ví dụ

□ Tìm ước lượng O-lớn cho các hàm sau:

□ $f(n) = 3n \log(n!) + (n^2 + 3) \log n$, n là số nguyên dương

□ $f(x) = (x + 1) \log(x^2 + 1) + 3x^2$

Omega-lớn & Theta-lớn

- **Khái niệm Omega-lớn:** Cho f và g là các hàm từ tập số nguyên hoặc tập số thực đến tập số thực. Ta nói $f(x)$ là $\Omega(g(x))$ nếu tồn tại hằng số C và k sao cho:

$$|f(x)| \geq C|g(x)|, \forall x > k$$

[Đọc là “ $f(x)$ là Ω -lớn (big- Ω) theo $g(x)$ ”]

- $f(x) = 8x^3 + 5x^2 + 7$ là $\Omega(g(x))$, với $g(x) = x^3$

Omega-lớn & Theta-lớn

- **Khái niệm Theta-lớn:** Cho f và g là các hàm từ tập số nguyên hoặc tập số thực đến tập số thực. Ta nói $f(x)$ là $\Theta(g(x))$ nếu $f(x)$ là $O(g(x))$ và $f(x)$ là $\Omega(g(x))$. Khi đó, $f(x)$ và $g(x)$ là cùng bậc với nhau. Tồn tại hằng số C_1 , C_2 và k sao cho:

$$C_1|g(x)| \leq |f(x)| \leq C_2|g(x)|, \forall x > k$$

[Đọc là “ $f(x)$ là Θ -lớn (big- Θ) theo $g(x)$ ”]

Omega-lớn & Theta-lớn

□ CMR

□ $f(n) = 1 + 2 + \dots + n$ là $\Theta(n^2)$

□ $f(n) = 3x^2 + 8x \log x$ là $\Theta(x^2)$

Độ phức tạp thuật toán

□ Độ phức tạp thời gian

- Số lượng phép toán được thực hiện bởi thuật toán với kích thước đầu vào cụ thể.
- Phép so sánh các số nguyên, cộng/nhân/chia các số nguyên, hoặc các phép toán cơ bản khác.
- Thuật toán *Tìm số lớn nhất* cần thực hiện $2(n-1) + 1 = 2n-1$ phép so sánh, nên Độ phức tạp thời gian là $\Theta(n)$

```
procedure max( $a_1, a_2, \dots, a_n$ : integers)
```

```
max :=  $a_1$ 
```

```
for  $i := 2$  to  $n$ 
```

```
    if  $max < a_i$  then  $max := a_i$ 
```

```
return  $max$ {max is the largest element}
```

Độ phức tạp thuật toán

□ Thuật toán tìm kiếm tuyến tính

□ Thực hiện $2n + 2$ phép so sánh trong trường hợp xấu nhất

○ $\Theta(n)$

procedure *linear search*(x : integer, $a_1, a_2, \dots, a_n$: distinct integers)

$i := 1$

while ($i \leq n$ and $x \neq a_i$)

$i := i + 1$

if $i \leq n$ **then** $location := i$

else $location := 0$

return $location$ { $location$ is the subscript of the term that equals x , or is 0 if x is not found}

Độ phức tạp tối nhất

□ Tìm độ phức tạp tối nhất của Thuật toán tìm kiếm nhị phân?

```
procedure binary search ( $x$ : integer,  $a_1, a_2, \dots, a_n$ : increasing integers)
 $i := 1$  { $i$  is left endpoint of search interval}
 $j := n$  { $j$  is right endpoint of search interval}
while  $i < j$ 
     $m := \lfloor (i + j) / 2 \rfloor$ 
    if  $x > a_m$  then  $i := m + 1$ 
    else  $j := m$ 
if  $x = a_i$  then  $location := i$ 
else  $location := 0$ 
return  $location$  { $location$  is the subscript  $i$  of the term  $a_i$  equal to  $x$ , or 0 if  $x$  is not found}
```


Độ phức tạp tối nhất

□ Tìm độ phức tạp tối nhất

□ Thuật toán sắp xếp nổi bọt

```
procedure bubblesort( $a_1, \dots, a_n$  : real numbers with  $n \geq 2$ )  
for  $i := 1$  to  $n - 1$   
    for  $j := 1$  to  $n - i$   
        if  $a_j > a_{j+1}$  then interchange  $a_j$  and  $a_{j+1}$   
{ $a_1, \dots, a_n$  is in increasing order}
```

□ Thuật toán sắp xếp chèn

```
procedure insertion sort( $a_1, a_2, \dots, a_n$ : real numbers with  $n \geq 2$ )  
for  $j := 2$  to  $n$   
     $i := 1$   
    while  $a_j > a_i$   
         $i := i + 1$   
     $m := a_j$   
    for  $k := 0$  to  $j - i - 1$   
         $a_{j-k} := a_{j-k-1}$   
     $a_i := m$   
{ $a_1, \dots, a_n$  is in increasing order}
```

Độ phức tạp trung bình

□ CMR: Thuật toán tìm kiếm tuyến tính có Độ phức tạp thời gian trung bình là $\Theta(n)$

```
procedure linear search(x: integer, a1, a2, ..., an: distinct integers)
  i := 1
  while (i ≤ n and x ≠ ai)
    i := i + 1
  if i ≤ n then location := i
  else location := 0
  return location{location is the subscript of the term that equals x, or is 0 if x is not found}
```

Họ thuật toán theo Độ phức tạp

Độ phức tạp	Thuật ngữ
$\Theta(1)$	Độ phức tạp Hằng
$\Theta(\log n)$	Độ phức tạp lô ga rít
$\Theta(n)$	Độ phức tạp tuyến tính
$\Theta(n \log n)$	Độ phức tạp lô ga tuyến tính
$\Theta(n^b)$	Độ phức tạp đa thức
$\Theta(b^n)$, where $b > 1$	Độ phức tạp lũy thừa (mũ)
$\Theta(n!)$	Độ phức tạp giai thừa

Độ phức tạp thực tế

□ Số lượng phép toán bit

- Giả sử mỗi phép toán bit mất 10^{-11} giây
- 10^{100} năm được thể hiện bằng dấu *

<i>Problem Size</i>	<i>Bit Operations Used</i>					
<i>n</i>	<i>log n</i>	<i>n</i>	<i>n log n</i>	<i>n²</i>	<i>2ⁿ</i>	<i>n!</i>
10	3×10^{-11} s	10^{-10} s	3×10^{-10} s	10^{-9} s	10^{-8} s	3×10^{-7} s
10^2	7×10^{-11} s	10^{-9} s	7×10^{-9} s	10^{-7} s	4×10^{11} yr	*
10^3	1.0×10^{-10} s	10^{-8} s	1×10^{-7} s	10^{-5} s	*	*
10^4	1.3×10^{-10} s	10^{-7} s	1×10^{-6} s	10^{-3} s	*	*
10^5	1.7×10^{-10} s	10^{-6} s	2×10^{-5} s	0.1 s	*	*
10^6	2×10^{-10} s	10^{-5} s	2×10^{-4} s	0.17 min	*	*

Các lớp bài toán

- ❑ Bài toán lần được (Tractable), không lần được (Intractable)
- ❑ Bài toán giải được (Solvable), không giải được (Unsolvable)
- ❑ Lớp P: Các bài toán lần được được
- ❑ Lớp NP (**n**ondeterministic **p**olynomial): Độ phức tạp (thời gian) đa thức không tất định
 - ❑ Ví dụ: Bài toán thoả mãn trong Lô gich mệnh đề
 - ❑ Các bài toán NP-đầy đủ
 - Hiện có 3000 bài toán NP-đầy đủ được biết, trong đó bài toán Thoả mãn là bài toán được chính mình thuộc lớp NP-đầy đủ đầu tiên.

P với NP

- ❑ Bài toán Thiên niên kỷ, do Viện Toán học Clay đề xuất, với giải thưởng 1,000,000 Đô la.
- ❑ $P = NP$?
 - ❑ “*Lớp các bài toán có thể kiểm tra được lời giải trong thời gian đa thức trùng với lớp các bài toán lần được.*”

Thuật toán Euclid

Số nguyên tố (prime)

□ **Định nghĩa:** Một số nguyên dương p lớn hơn 1 và chỉ chia hết cho 1 và chính nó được gọi là một **số nguyên tố**.

□ **Ví dụ:** 2, 3, 5, 7,...

$1|2$ và $2|2$; $1|3$ và $3|3$;...

Số nguyên tố tiếp theo sau 7?

- 11

Tiếp theo?

- 13

Số nguyên tố

□ **Định nghĩa:** Một số nguyên dương lớn hơn 1 và không phải là số nguyên tố gọi là **một hợp số (composite number)**.

□ **Ví dụ:** 4, 6, 8, 9, ...

Vì sao?

- $2|4$

Vì sao 6 là hợp số?

Định lý cơ bản của số học (arithmetic)

□ Định lý Cơ bản của Số học:

- Bất cứ số nguyên dương lớn hơn 1 nào đều có thể được biểu diễn dưới dạng tích của các số nguyên tố.

□ Ví dụ:

- $12 = 2 \times 2 \times 3$
- $21 = 3 \times 7$

□ Quy trình tìm ra các thừa số: **phân tích thừa số (factorization)**

Số nguyên tố và Hợp số

□ Phân tích thừa số nguyên tố của các hợp số:

- $100 = 2 \times 2 \times 5 \times 5 = 2^2 \times 5^2$
- $99 = 3 \times 3 \times 11 = 3^2 \times 11$

□ Câu hỏi quan trọng:

- Làm thế nào để xác định một số là số nguyên tố hay hợp số?

Số nguyên tố và Hợp số

❑ Làm thế nào để xác định một số là số nguyên tố hay hợp số?

❑ **Phương pháp đơn giản (1):**

- Để xác định một số n có phải là số nguyên tố không, ta có thể kiểm tra với **một số x** nào đó $x < n$ và n chia hết cho x . **Nếu đúng thì nó là hợp số.**
- Nếu ta kiểm tra hết **tất cả các số $x < n$** và không tìm thấy một số chia nào thì n là số nguyên tố.
- **Ví dụ:**
Giả sử ta muốn kiểm tra 17 có phải là số nguyên tố?
Phương pháp trên yêu cầu ta phải kiểm tra:
2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16

Số nguyên tố và Hợp số

❑ Ví dụ Phương pháp 1:

Giả sử ta muốn kiểm tra 17 có phải là số nguyên tố?

Phương pháp trên yêu cầu ta phải kiểm tra:

2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16

❑ Đây có phải là cách tốt nhất không?

- **Không.** Vấn đề ở đây là chúng ta phải kiểm tra tất cả các số và điều này là không cần thiết.

❑ Ý tưởng:

- Tất cả các hợp số đều có thể được phân tích thành tích của các thừa số nguyên tố. Do đó, chỉ cần kiểm tra các số nguyên tố $x < n$ để xác định n có phải số nguyên tố hay không.

Số nguyên tố và Hợp số

❑ Làm thế nào để xác định một số là số nguyên tố hay hợp số?

❑ Phương pháp 2:

- Để xác định số n có là số nguyên tố hay không, ta có thể kiểm tra có tồn tại **số nguyên tố x** nào đó, $x < n$ và n chia hết cho x hay không. **Nếu có thì n là hợp số.**
- Nếu ta kiểm tra tất cả các **số nguyên tố $x < n$** và không tìm thấy số chia nào thì **n là số nguyên tố.**
- **Ví dụ:** 31 có phải số nguyên tố không?
Kiểm tra nó có chia hết cho 2, 3, 5, 7, 11, 13, 17, 23, 29 không?
Nó là số nguyên tố.

Số nguyên tố và Hợp số

□ Ví dụ Phương pháp 2:

- 91 có phải là số nguyên tố không?
- Các số nguyên tố dễ biết: 2, 3, 5, 7, 11, 13, 17, 19,...
- Nhưng có bao nhiêu số nguyên tố nhỏ hơn 91?

□ Lưu ý:

- Nếu n tương đối nhỏ, phép kiểm tra là tốt vì ta có thể lập (ghi nhớ) tất cả những số nguyên tố nhỏ trước nó.
- Nhưng nếu n lớn thì sẽ có những số nguyên tố lớn hơn và không dễ thấy.

Số nguyên tố và Hợp số

□ **Định lý:** Nếu n là hợp số thì n có một số chia là số nguyên tố nhỏ hơn hoặc bằng $\sqrt{n}$.

□ Chứng minh:

- Nếu n là hợp số, thì nó có số chia a là số nguyên dương sao cho $1 < a < n$ theo định nghĩa. Điều này có nghĩa là $n = ab$, với b là một số nguyên lớn hơn 1.
- Giả sử $a > \sqrt{n}$ và $b > \sqrt{n}$. Thì $ab > \sqrt{n}\sqrt{n} = n$, trái với giả thiết. Vì vậy, hoặc là $a \leq \sqrt{n}$ hoặc $b \leq \sqrt{n}$.
- Do đó, n có số chia nhỏ hơn $\sqrt{n}$.
- Dựa vào Định lý Cơ bản của Số học, số chia này hoặc là số nguyên tố, hoặc là tích của các số nguyên tố.
 - Trong cả 2 trường hợp, n đều có một số chia là số nguyên tố nhỏ hơn $\sqrt{n}$.

Số nguyên tố và Hợp số

- ❑ **Định lý:** Nếu n là hợp số thì n có một số chia là số nguyên tố nhỏ hơn hoặc bằng $\sqrt{n}$.
- ❑ **Phương pháp 3:**
 - Để xác định n có phải số nguyên tố hay không, ta có thể kiểm tra có tồn tại số chia x nào đó là số nguyên tố và $x < \sqrt{n}$ hay không.
- ❑ **Ví dụ:** 101 có phải là số nguyên tố không?
 - Các số nguyên tố nhỏ hơn $\sqrt{101} = 10.xxx$ là 2, 3, 5, 7
 - 101 không chia hết cho các số trên
 - Do đó, 101 là số nguyên tố
- ❑ **Ví dụ:** 91 có phải là số nguyên tố không?
 - Các số nguyên tố nhỏ hơn $\sqrt{91}$ là 2, 3, 5, 7
 - 91 chia hết cho 7
 - Do đó, 91 là hợp số.

Số nguyên tố

- ❑ Câu hỏi đặt ra là có tất cả bao nhiêu số nguyên tố?
- ❑ **Định lý:** Có vô số số nguyên tố.
- ❑ **Chứng minh bởi Euclid:**
 - Chứng minh phản chứng: Giả sử có hữu hạn các số nguyên tố: $p_1, p_2, \dots, p_n$
 - Đặt $Q = p_1 p_2 \dots p_n + 1$ là một số
 - Q không chia hết cho bất cứ số nào trong số $p_1, p_2, \dots, p_n$
 - Điều này là vô lý vì đã giả sử rằng ta đã liệt kê hết các số nguyên tố.

Ước chung lớn nhất

□ **Định nghĩa:** Cho a và b là các số nguyên, không đồng thời bằng 0. Số nguyên lớn nhất d sao cho $d|a$ và $d|b$ được gọi là **ước chung lớn nhất (greatest common divisor)** của a và b . Ước chung lớn nhất được ký hiệu là **$\gcd(a, b)$** .

□ **Ví dụ:**

- $\gcd(24, 36) = ?$
Kiểm tra 2, 3, 4, 6, 12 $\Rightarrow \gcd(24, 36) = 12$
- $\gcd(11, 23) = ?$

Ước chung lớn nhất

□ Tìm gcd sử dụng phân tích thừa số

- Đặt $a = p_1^{a_1} p_2^{a_2} p_3^{a_3} \dots p_k^{a_k}$ và $b = p_1^{b_1} p_2^{b_2} p_3^{b_3} \dots p_k^{b_k}$
- $\gcd(a, b) = p_1^{\min(a_1, b_1)} p_2^{\min(a_2, b_2)} p_3^{\min(a_3, b_3)} \dots p_k^{\min(a_k, b_k)}$

□ Ví dụ:

- $\gcd(24, 36) = ?$
- $24 = 2 \times 2 \times 2 \times 3 = 2^3 \times 3$
- $36 = 2 \times 2 \times 3 \times 3 = 2^2 \times 3^2$
- $\gcd(24, 36) = 2^2 \times 3 = 12$

Bội chung nhỏ nhất

□ **Định nghĩa:** Đặt a và b là các số nguyên dương. **Bội chung nhỏ nhất (least common multiplier)** của a và b là số nguyên dương nhỏ nhất chia hết cho cả a và b . Bội chung nhỏ nhất được ký hiệu là $\text{lcm}(a, b)$.

□ **Ví dụ:**

- $\text{lcm}(12, 9) = ?$
- Hãy đưa ra một bội chung: ...

Bội chung nhỏ nhất

□ **Định nghĩa:** Đặt a và b là các số nguyên dương. **Bội chung nhỏ nhất (least common multiplier)** của a và b là số nguyên dương nhỏ nhất chia hết cho cả a và b . Bội chung nhỏ nhất được ký hiệu là $\text{lcm}(a, b)$.

□ **Ví dụ:**

- $\text{lcm}(12, 9) = ?$
- Hãy đưa ra một bội chung: ... $12 \times 9 = 108$
- Ta có thể tìm được số nhỏ hơn không?
- Có. Thử 36. Cả hai số 12 và 9 đều được chia hết bởi 36.

Bội chung nhỏ nhất

□ Tìm lcm sử dụng phân tích thừa số

- Đặt $a = p_1^{a_1} p_2^{a_2} p_3^{a_3} \dots p_k^{a_k}$ và $b = p_1^{b_1} p_2^{b_2} p_3^{b_3} \dots p_k^{b_k}$
- $\text{lcm}(a, b) = p_1^{\max(a_1, b_1)} p_2^{\max(a_2, b_2)} p_3^{\max(a_3, b_3)} \dots p_k^{\max(a_k, b_k)}$

□ Ví dụ:

- $\text{lcm}(12, 9) = ?$
- $12 = 2 \times 2 \times 3 = 2^2 \times 3$
- $9 = 3 \times 3 = 3^2$
- $\text{lcm}(12, 9) = 2^2 \times 3^2 = 4 \times 9 = 36$

Thuật toán Euclid

□ Tìm ước chung lớn nhất đòi hỏi phân tích thừa số

- $a = p_1^{a_1} p_2^{a_2} p_3^{a_3} \dots p_k^{a_k}$ và $b = p_1^{b_1} p_2^{b_2} p_3^{b_3} \dots p_k^{b_k}$
- $\gcd(a, b) = p_1^{\min(a_1, b_1)} p_2^{\min(a_2, b_2)} p_3^{\min(a_3, b_3)} \dots p_k^{\min(a_k, b_k)}$
- Phân tích thừa số có thể rất tốn thời gian vì ta phải tìm tất cả các thừa số của 2 số nguyên có thể rất lớn.
- May mắn là có một phương pháp hiệu quả hơn để tính gcd, là **thuật toán Euclid**.
 - Phương pháp được biết đến từ xa xưa và được đặt tên theo nhà toán học Hy Lạp Euclid.

Thuật toán Euclid

□ Ta muốn tìm $\gcd(287, 91)$.

- Đầu tiên, chia số lớn (287) cho số bé (91)
- Ta có $287 = 3 \times 91 + 14$

1) Bất cứ số chia nào của 91 và 287 đều là số chia của 14:

- $287 - 3 \times 91 = 14$
- Tại sao? $[ak - cbk] = r \Rightarrow (a - cb)k = r \Rightarrow (a - cb) = \frac{r}{k}$
(phải là số nguyên, vì vậy r chia hết cho k).

2) Bất cứ số chia nào của 91 và 14 đều là số chia của 287:

- Tại sao? $287 = 3bk + dk \Rightarrow 287 = k(3b + d) \Rightarrow \frac{287}{k} = (3b + d) \leftarrow \frac{287}{k}$ là số nguyên.
- Do đó, $\gcd(287, 91) = \gcd(91, 14)$

Thuật toán Euclid

- ❑ Ta biết rằng $\gcd(287, 91) = \gcd(91, 14)$
- ❑ Có thể áp dụng lại cách đó:
 - $\gcd(91, 14)$
 - $91 = 14 \times 6 + 7$
- ❑ Vì vậy
 - $\gcd(91, 14) = \gcd(14, 7)$
- ❑ Áp dụng 1 lần nữa:
 - $\gcd(14, 7) = 7$
 - Dễ giải
- ❑ Kết quả: $\gcd(287, 91) = \gcd(91, 14) = \gcd(14, 7) = 7$

Thuật toán Euclid

□ Ví dụ:

- Tìm ước chung lớn nhất của 666 và 558
 - $\gcd(666, 558)$
 $= \gcd(558, 108)$
 $= \gcd(108, 18)$
- $$666 = 1 \times 558 + 108$$
$$558 = 5 \times 108 + 18$$
$$108 = 6 \times 18 + 0 = 18$$

Thuật toán Euclid

□ Ví dụ

- Tìm ước chung lớn nhất của 286 và 503:
 - $\gcd(503, 286)$
 $= \gcd(286, 217)$
 $= \gcd(217, 69)$
 $= \gcd(69, 10)$
 $= \gcd(10, 9)$
 $= \gcd(9, 1) = 1$
- $503 = 1 \times 286 + 217$
 $286 = 1 \times 217 + 69$
 $217 = 3 \times 69 + 10$
 $69 = 6 \times 10 + 9$
 $10 = 1 \times 9 + 1$